CMDA-4654
Project 1

Kathleen Reece, Curtis Wong, and Abia Zahir

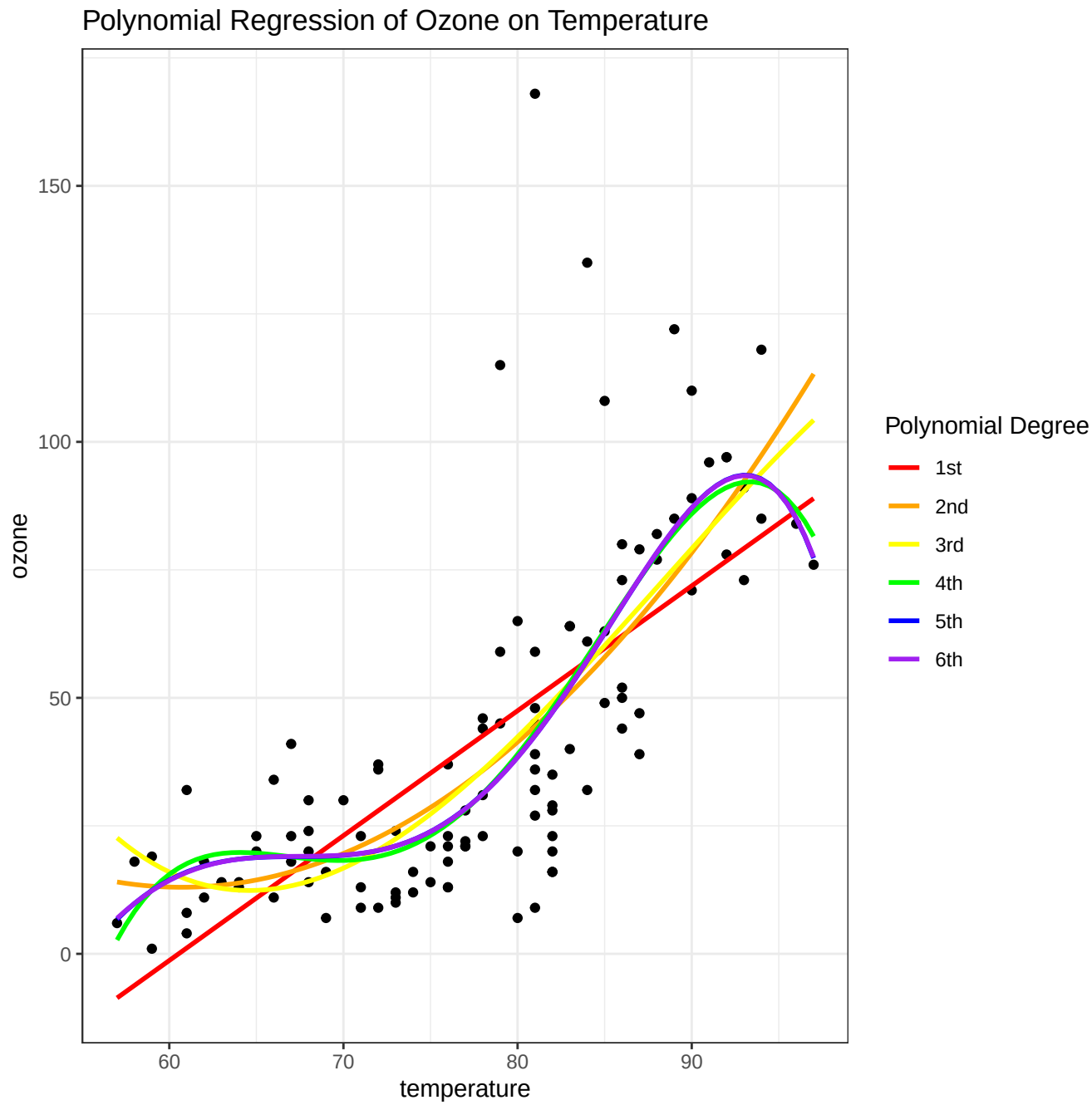
April 08, 2026

Problem 1

```r
# Loading in the data set
load("ozone.RData")
data("ozone")
```

Part 1

```r
# Fit models and overlay on top of each other
# For easy comparison
ggplot(ozone, aes(x = temperature, y = ozone)) + theme_bw() + geom_point() +
  geom_smooth(method = "lm", se = F, aes(color = "1st")) +
  geom_smooth(method = "lm", formula = "y ~ poly(x, 2)", se = F, aes(color = "2nd")) +
  geom_smooth(method = "lm", formula = "y ~ poly(x, 3)", se = F, aes(color = "3rd")) +
  geom_smooth(method = "lm", formula = "y ~ poly(x, 4)", se = F, aes(color = "4th")) +
  geom_smooth(method = "lm", formula = "y ~ poly(x, 5)", se = F, aes(color = "5th")) +
  geom_smooth(method = "lm", formula = "y ~ poly(x, 6)", se = F, aes(color = "6th")) +
  scale_color_manual(name = "Polynomial Degree",
                     breaks = c("1st", "2nd", "3rd", "4th", "5th", "6th"),
                     values = c("red", "orange", "yellow", "green", "blue", "purple")) +
  labs(title = "Polynomial Regression of Ozone on Temperature")
```



From the graph, it seems like the 2nd-degree fit is the best. However, none of the degrees seem to give a satisfactory fit.

Part 2

```
# Where we'll store all the LOESS fits we're making
models_p1 <- vector(mode = "list", length = 22)
# Staring span value
span_val <- 0.25
# Fit all the degree-1 models
for (i in 1:11) {
  models_p1[[i]] <- myloess(x_train = ozone$temperature, y_train = ozone$ozone,
                        span = span_val, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Reset span
span_val <- 0.25
# Fit all the degree-2 models
for (i in 12:22) {
```

```r
  models_p1[[i]] <- myloess(x_train = ozone$temperature, y_train = ozone$ozone,
                            span = span_val, degree = 2, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Put the span, degree, and MSE into dataframe for easy table making
model_df <- dplyr::bind_rows( lapply(models_p1, function(m) {
  data.frame(
      span = m$span,
      degree = m$degree,
      MSE = m$MSE
    )
  })
)
# Output table
knitr::kable(
  model_df,
  caption = "LOESS Fits on Ozone Data"
)
```

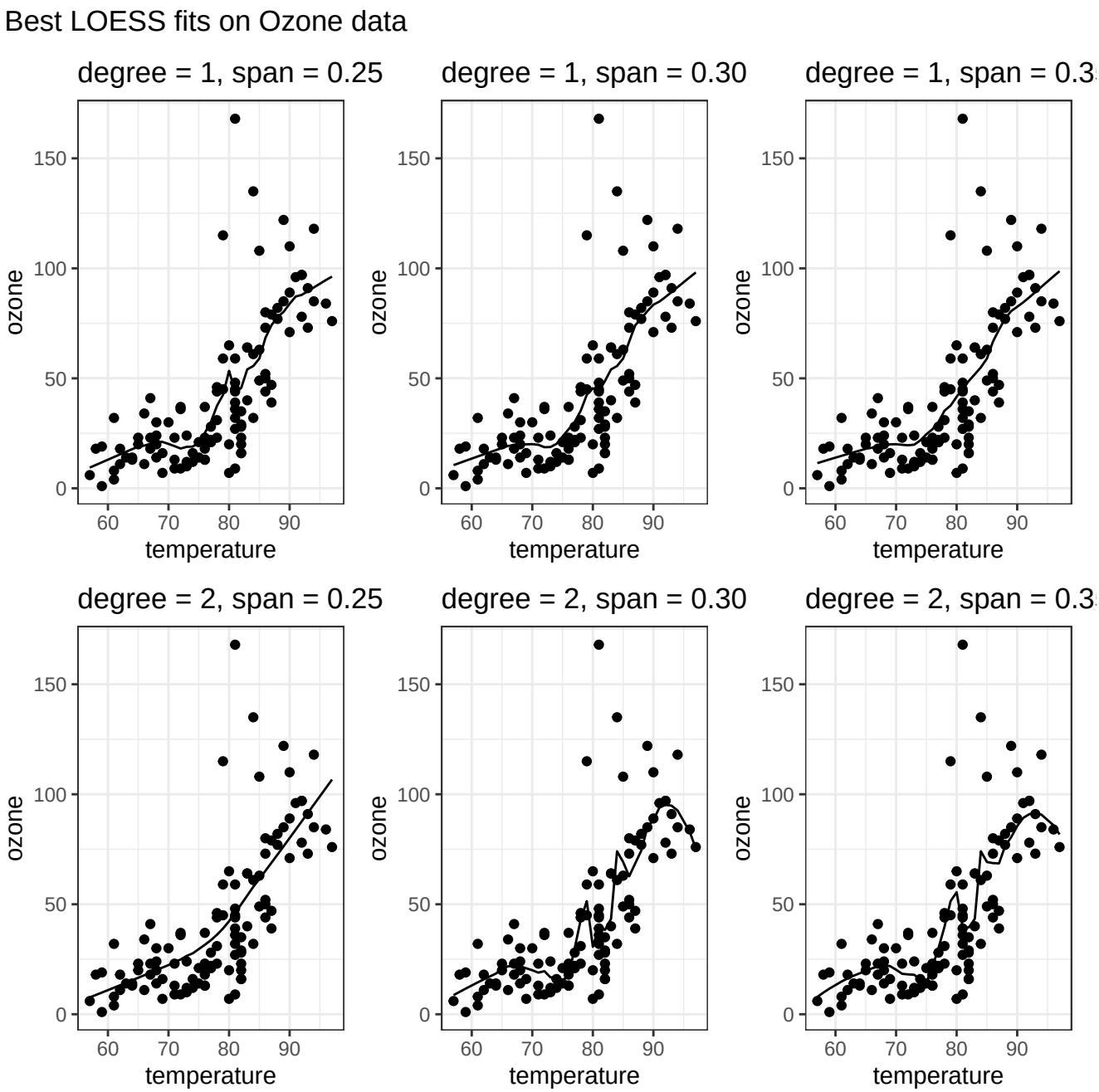
Table 1: LOESS Fits on Ozone Data

span	degree	MSE
0.25	1	467.6651
0.30	1	470.7391
0.35	1	476.7014
0.40	1	485.8182
0.45	1	482.4721
0.50	1	484.9289
0.55	1	486.0865
0.60	1	487.3726
0.65	1	489.6556
0.70	1	491.0533
0.75	1	494.7735
0.25	2	403.1341
0.30	2	427.5517
0.35	2	428.5621
0.40	2	443.4471
0.45	2	468.4704
0.50	2	471.9893
0.55	2	473.4295
0.60	2	474.5741
0.65	2	475.6940
0.70	2	477.2696
0.75	2	480.6660

From the table, the best fits with a degree of 1 occur for span values of 0.25, 0.30, and 0.35, and the best fits with a degree of 2 occur for span values of 0.25, 0.30, and 0.35 as well.

```r
# Use the plots from the myloess function, but add titles
p1 <- models_p1[[1]]$loessplot + labs(title = "degree = 1, span = 0.25",
                                      x = "temperature", y = "ozone")
p2 <- models_p1[[2]]$loessplot + labs(title = "degree = 1, span = 0.30",
                                      x = "temperature", y = "ozone")
p3 <- models_p1[[3]]$loessplot + labs(title = "degree = 1, span = 0.35",
                                      x = "temperature", y = "ozone")
p4 <- models_p1[[11]]$loessplot + labs(title = "degree = 2, span = 0.25",
                                      x = "temperature", y = "ozone")
p5 <- models_p1[[12]]$loessplot + labs(title = "degree = 2, span = 0.30",
                                      x = "temperature", y = "ozone")
p6 <- models_p1[[13]]$loessplot + labs(title = "degree = 2, span = 0.35",
```

```
                                    x = "temperature", y = "ozone")
# Use patchwork library to plot all together
(p1 + p2 + p3) / (p4 + p5 + p6) + plot_annotation(title = "Best LOESS fits on Ozone data")
```



Based the plots, I would consider all 6 of the models to be better fits than any of the polynomial fits. However, I would also consider almost all of the curves to be somewhat overfitted to the data. In particular, the curves where the local fits used a degree 2 polynomial are very jagged, seemingly in response to some small shifts in the data; the degree-1 models with spans of 0.25 and 0.30 also show some of this jaggedness, though on a lesser degree. Of all of the "best" fits, the one with the least signs of overfitting is the LOESS curve with degree = 1 and span = 0.35.

Part 3

```
# Where we'll store all the LOESS fits we're making
baser_models_p1 <- vector(mode = "list", length = 22)
# Staring span value
span_val <- 0.25
# Fit all the degree-1 models
for (i in 1:11) {
  baser_models_p1[[i]] <- loess(ozone ~ temperature, data = ozone, span = span_val, degree = 1)
```

```r
  span_val <- span_val + 0.05
}
# Reset span
span_val <- 0.25
# Fit all the degree-2 models
for (i in 12:22) {
  baser_models_p1[[i]] <- loess(ozone ~ temperature, data = ozone, span = span_val)
  span_val <- span_val + 0.05
}
# Put the span, degree, and RSE into dataframe for easy table making
baser_model_df <- dplyr::bind_rows(lapply(baser_models_p1, function(m) {
  data.frame(
      span = m$pars$span,
      degree = m$pars$degree,
      RSE = m$s
    )
  })
)
# Output table
knitr::kable(
  baser_model_df,
  caption = "Loess Fits on Ozone Data, with loess()"
)
```

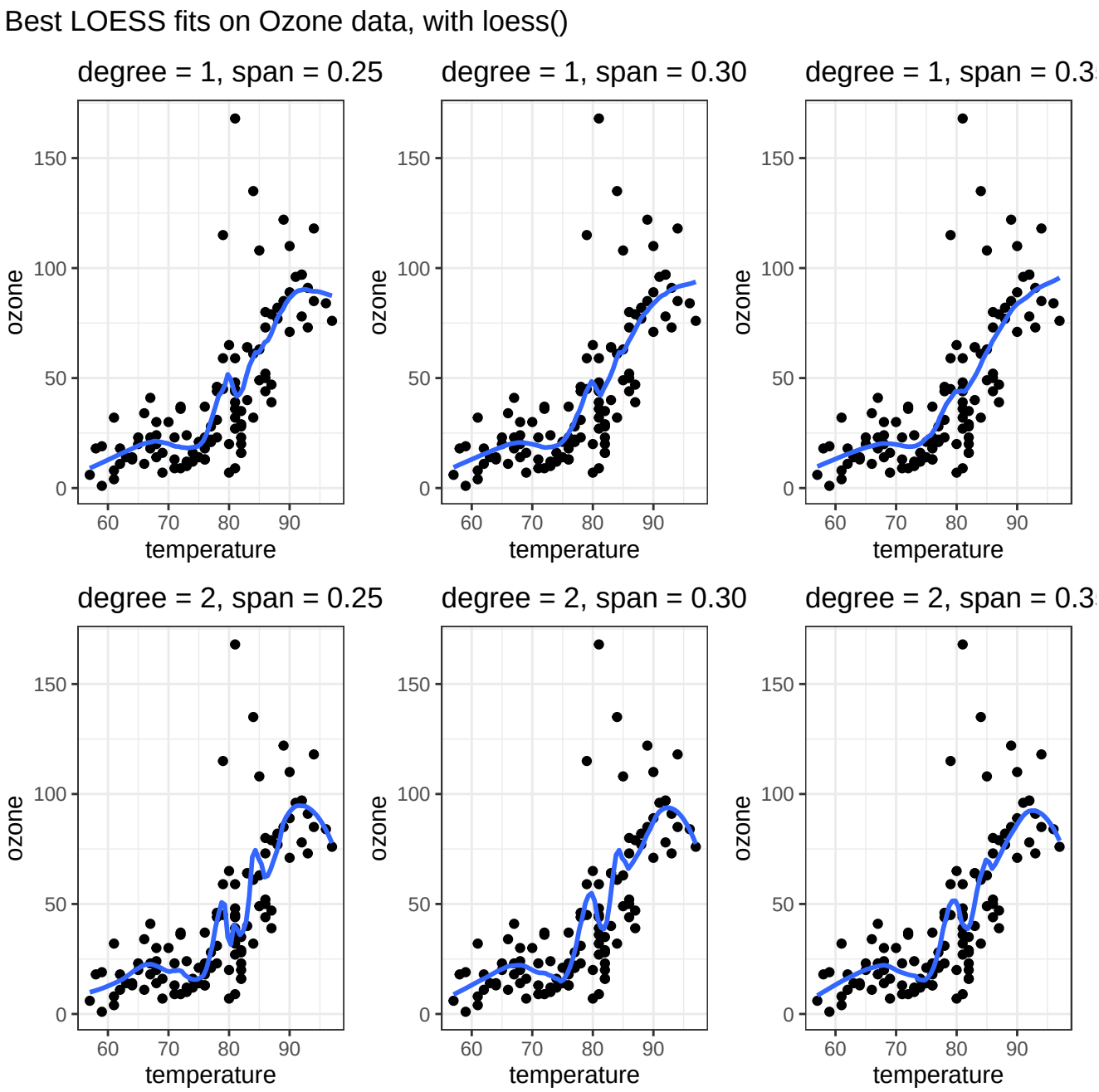
Table 2: Loess Fits on Ozone Data, with loess()

span	degree	RSE
0.25	1	22.03363
0.30	1	22.13759
0.35	1	22.20393
0.40	1	22.35919
0.45	1	22.42632
0.50	1	22.45759
0.55	1	22.42290
0.60	1	22.45481
0.65	1	22.44426
0.70	1	22.44382
0.75	1	22.48305
0.25	2	21.35833
0.30	2	21.68588
0.35	2	21.67665
0.40	2	21.83337
0.45	2	22.12619
0.50	2	22.19365
0.55	2	22.27206
0.60	2	22.33931
0.65	2	22.36149
0.70	2	22.35995
0.75	2	22.35047

Although the built in `loess()` function uses RSE instead of MSE, it agrees on the best 3 models of each degree.

```r
# Use the plots from the myloess function, but add titles
g1 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.25, se = F, method.args = list(degree = 1)) +
  labs(title = "degree = 1, span = 0.25")
g2 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.3, se = F, method.args = list(degree = 1)) +
  labs(title = "degree = 1, span = 0.30")
```

```r
g3 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.35, se = F, method.args = list(degree = 1)) +
  labs(title = "degree = 1, span = 0.35")
g4 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.25, se = F, method.args = list(degree = 2)) +
  labs(title = "degree = 2, span = 0.25")
g5 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.3, se = F, method.args = list(degree = 2)) +
  labs(title = "degree = 2, span = 0.30")
g6 <- ggplot(ozone, aes(x = temperature, y = ozone)) + geom_point() + theme_bw() +
  geom_smooth(method = "loess", span = 0.35, se = F, method.args = list(degree = 2)) +
  labs(title = "degree = 2, span = 0.35")
# Use patchwork library to plot all together
(g1 + g2 + g3) / (g4 + g5 + g6) + plot_annotation(title = "Best LOESS fits on Ozone data, with loess()")
```



The plots look very similar to those from the `myloess()` function. The only difference is that the curves look a tad smoother.

Problem 2

```r
# Loading the data
library(MASS)
data("mcycle")
load("mcycle_test.RData")
data("mcycle_test")
```

Part 1

We can use the same fitting-and-plotting process as in part 3 of problem 1:

```r
# Where we'll store all the LOESS fits we're making
models_p2 <- vector(mode = "list", length = 22)
# Staring span value
span_val <- 0.25
# Fit all the degree-1 models
for (i in 1:11) {
  models_p2[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                        span = span_val, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Reset span
span_val <- 0.25
# Fit all the degree-2 models
for (i in 12:22) {
  models_p2[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                        span = span_val, degree = 2, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Put the span, degree, and MSE into dataframe for easy table making
model_df <- dplyr::bind_rows( lapply(models_p2, function(m) {
  data.frame(
      span = m$span,
      degree = m$degree,
      MSE = m$MSE
    )
  })
)
# Output table
knitr::kable(
  model_df,
  caption = "LOESS Fits on mcycle Data"
)
```

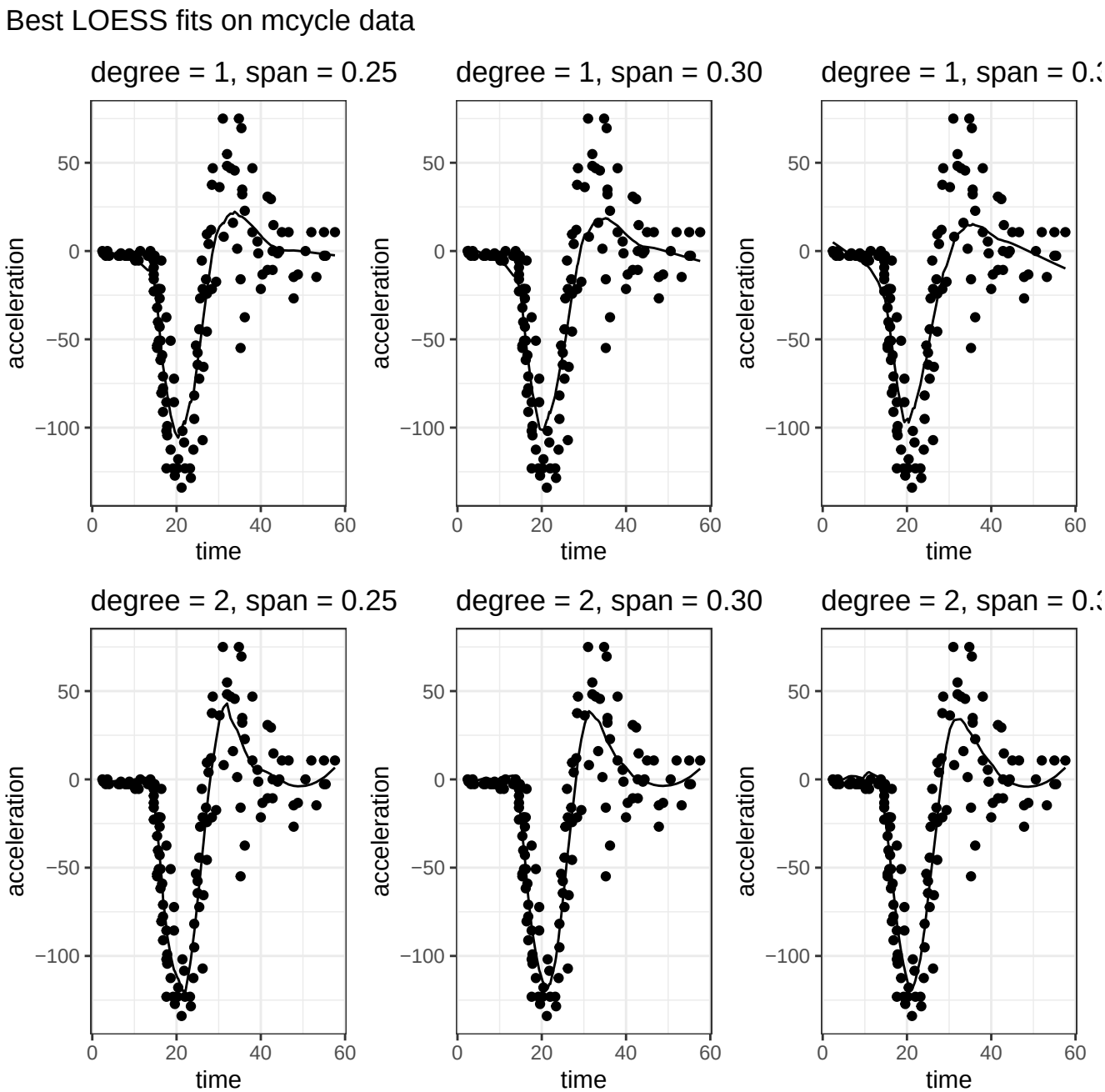
Table 3: LOESS Fits on mcycle Data

span	degree	MSE
0.25	1	531.5884
0.30	1	586.2357
0.35	1	664.9955
0.40	1	774.5691
0.45	1	878.0587
0.50	1	1019.2968
0.55	1	1165.2179
0.60	1	1288.7858
0.65	1	1395.3525
0.70	1	1498.9039
0.75	1	1596.6074
0.25	2	455.4035

span	degree	MSE
0.30	2	461.9929
0.35	2	475.7662
0.40	2	504.5805
0.45	2	549.4119
0.50	2	601.6663
0.55	2	674.9693
0.60	2	753.7812
0.65	2	875.4644
0.70	2	1020.6186
0.75	2	1154.4972

For degree = 1, the best span values are 0.25, 0.30, and 0.35. For degree = 2, the best span values are also 0.25, 0.30, and 0.35.

```
# Use the plots from the myloess function, but add titles
p1 <- models_p2[[1]]$loessplot + labs(title = "degree = 1, span = 0.25",
                                      x = "time", y = "acceleration")
p2 <- models_p2[[2]]$loessplot + labs(title = "degree = 1, span = 0.30",
                                      x = "time", y = "acceleration")
p3 <- models_p2[[3]]$loessplot + labs(title = "degree = 1, span = 0.35",
                                      x = "time", y = "acceleration")
p4 <- models_p2[[12]]$loessplot + labs(title = "degree = 2, span = 0.25",
                                      x = "time", y = "acceleration")
p5 <- models_p2[[13]]$loessplot + labs(title = "degree = 2, span = 0.30",
                                      x = "time", y = "acceleration")
p6 <- models_p2[[14]]$loessplot + labs(title = "degree = 2, span = 0.35",
                                      x = "time", y = "acceleration")
# Use patchwork library to plot all together
(p1 + p2 + p3) / (p4 + p5 + p6) + plot_annotation(title = "Best LOESS fits on mcycle data")
```



All of the plots look pretty good, but in my opinion, the models with degree = 2 do a better job capturing the true shape than the models with degree = 1, especially for the flat section of points at the far left and the large curve in the middle of the plot. Also, all of the best models with degree = 2 have lower MSE values than all of the best models with degree = 1. Thus, I would say LOESS with degree = 2 and a span of 0.25, 0.30, or 0.35 provides the best fits to the data.

Part 2

```
# Where we'll store all the LOESS fits we're making
test_models <- vector(mode = "list", length = 24)
# Staring span value
span_val <- 0.1
# Fit all the degree-1 models from span = 0.1 to 0.65, incrementing by 0.05
for (i in 1:12) {
  test_models[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                              x_test = mcycle_test$times, y_test = mcycle_test$accel,
                        span = span_val, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Reset span
```

```r
span_val <- 0.1
# Fit all the degree-2 models from span = 0.1 to 0.65, incrementing by 0.05
for (i in 13:24) {
  test_models[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                              x_test = mcycle_test$times, y_test = mcycle_test$accel,
                        span = span_val, degree = 2, show.plot = FALSE)
  span_val <- span_val + 0.05
}
# Put the span, degree, and MSE into a dataframe for easy table making
test_model_df <- dplyr::bind_rows( lapply(test_models, function(m) {
  data.frame(
      span = m$span,
      degree = m$degree,
      MSE = m$MSE
    )
  })
)
# Output table
knitr::kable(
  test_model_df,
  caption = "LOESS Fits on mcycle testing data"
)
```

Table 4: LOESS Fits on mcycle testing data

span	degree	MSE
0.10	1	74.45679
0.15	1	77.37762
0.20	1	110.69368
0.25	1	170.22404
0.30	1	241.86712
0.35	1	359.60204
0.40	1	502.30695
0.45	1	645.47300
0.50	1	836.33576
0.55	1	1025.62133
0.60	1	1193.74650
0.65	1	1342.40226
0.10	2	104.61616
0.15	2	79.17058
0.20	2	71.77071
0.25	2	74.31634
0.30	2	80.62825
0.35	2	95.67421
0.40	2	141.93397
0.45	2	193.36285
0.50	2	272.94385
0.55	2	357.88760
0.60	2	433.95078
0.65	2	566.77605

For degree = 1, the optimal span that minimizes MSE seems to be around 0.10., and for degree = 2, the optimal span that minimizes MSE seems to be around 0.20. We can test spans closer to these values:

```r
# Where we'll store all the LOESS fits we're making
test_models2 <- vector(mode = "list", length = 18)
# Staring span value
span_val <- 0.06
# Fit all the degree-1 models from span = 0.06 to 0.14, incrementing by 0.01
```

```r
for (i in 1:9) {
  test_models2[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                               x_test = mcycle_test$times, y_test = mcycle_test$accel,
                        span = span_val, show.plot = FALSE)
  span_val <- span_val + 0.01
}
# Reset span
span_val <- 0.16
# Fit all the degree-2 models from span = 0.16 to 0.24, incrementing by 0.01
for (i in 10:18) {
  test_models2[[i]] <- myloess(x_train = mcycle$times, y_train = mcycle$accel,
                               x_test = mcycle_test$times, y_test = mcycle_test$accel,
                        span = span_val, degree = 2, show.plot = FALSE)
  span_val <- span_val + 0.01
}
# Put the span, degree, and MSE into a dataframe for easy table making
test_model_df2 <- dplyr::bind_rows( lapply(test_models2, function(m) {
  data.frame(
      span = m$span,
      degree = m$degree,
      MSE = m$MSE
    )
  })
)
# Output table
knitr::kable(
  test_model_df2,
  caption = "LOESS Fits on mcycle testing data"
)
```

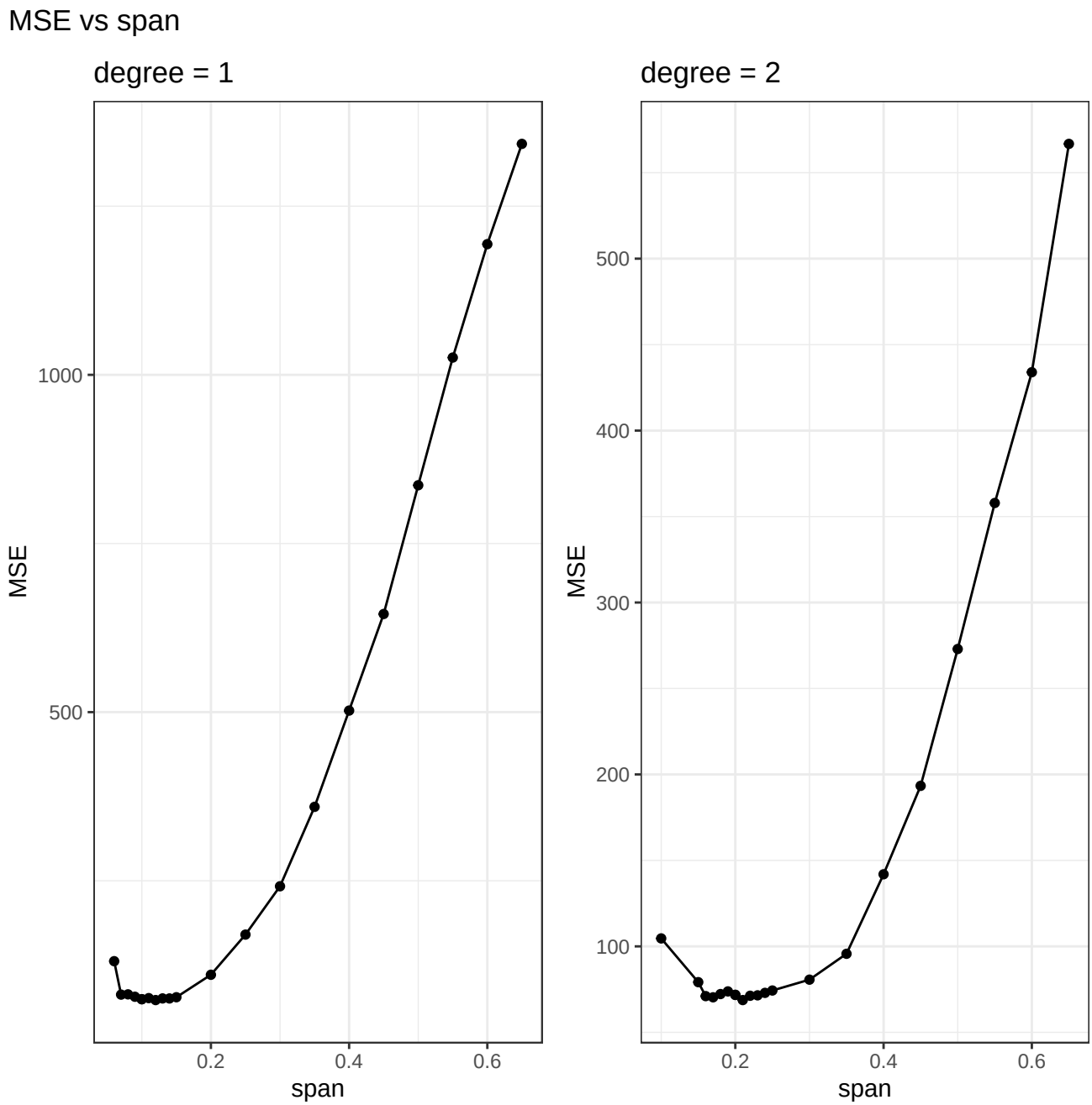
Table 5: LOESS Fits on mcycle testing data

span	degree	MSE
0.06	1	130.81066
0.07	1	81.26522
0.08	1	81.64983
0.09	1	78.14124
0.10	1	74.45679
0.11	1	76.07479
0.12	1	73.17798
0.13	1	75.59776
0.14	1	75.58498
0.16	2	71.05422
0.17	2	70.35637
0.18	2	72.25160
0.19	2	73.78643
0.20	2	71.77071
0.21	2	68.75884
0.22	2	71.25890
0.23	2	71.51535
0.24	2	72.94028

From our closer look, the optimal span when degree = 1 is 0.12, and the optimal span when degree = 2 is 0.21. Interestingly, there is a difference in relationship between spans and MSE when using only the training data vs the testing and training data. For the former, the MSE increased as span increased; for the latter, the relationship seems to be more of a curve that reaches a mimimum MSE before rapidly increasing toward infinity. We can make a graph to further understand this relationship:

```r
# Combine dataframes containing the spans and MSEs into one and split
# them again by degree
```

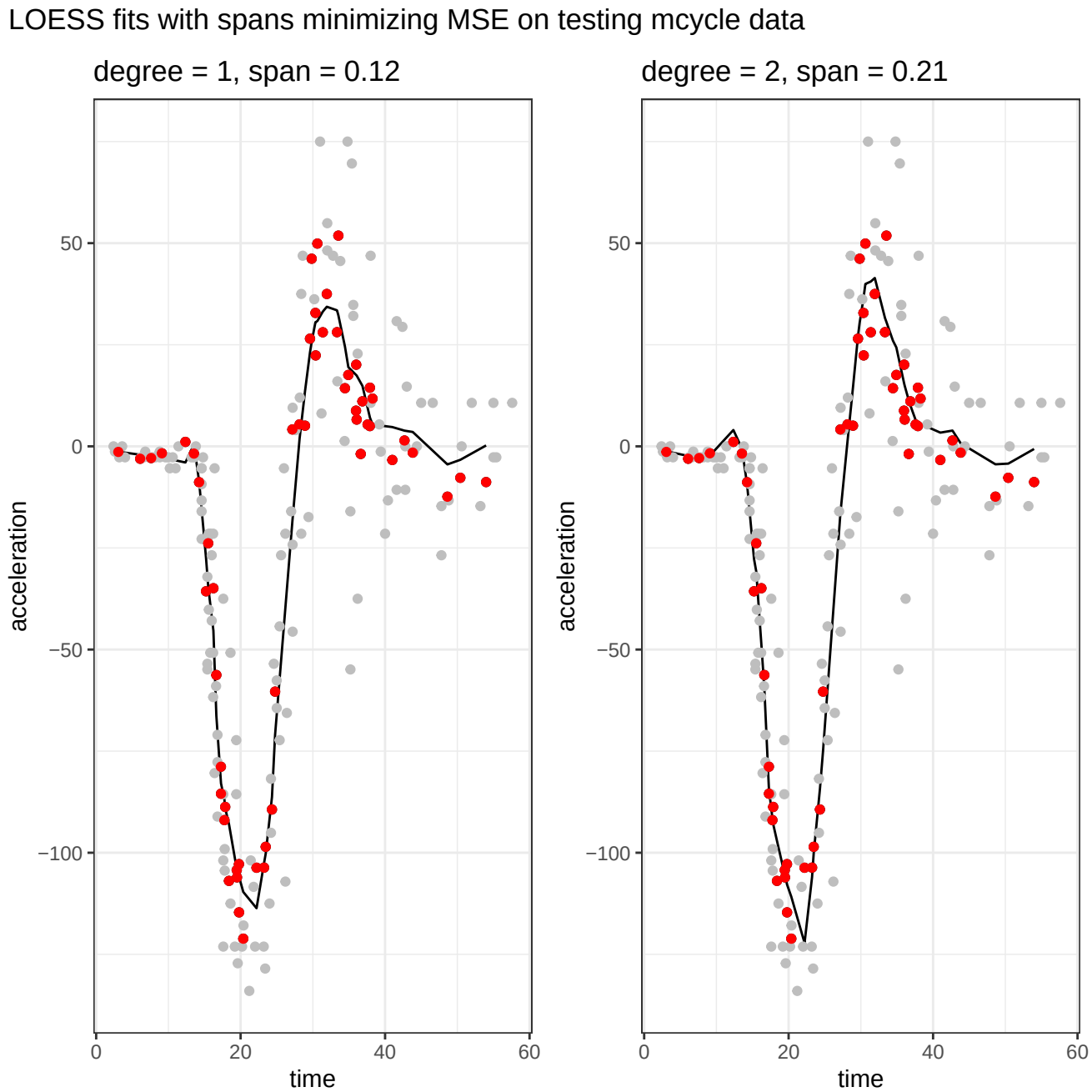
```r
all_spans <- rbind(test_model_df, test_model_df2)
spans1 <- filter(all_spans, degree == 1)
spans2 <- filter(all_spans, degree == 2)
# Graph MSE vs span for each degree
span_curve1 <- ggplot(data = spans1, aes(x = span, y = MSE)) + geom_point() +
  geom_line(aes(group = 1)) + theme_bw() + labs(title = "degree = 1")
span_curve2 <- ggplot(data = spans2, aes(x = span, y = MSE)) + geom_point() +
  geom_line(aes(group = 1)) + theme_bw() + labs(title = "degree = 2")
span_curve1 + span_curve2 + plot_annotation(title = "MSE vs span")
```



We can plot the data (testing and training) with the curves with the optimal spans for each degree

```r
# Use myloess plots as base, then recolor the points and add labels
best_span1 <- test_models2[[7]]$loessplot +
  geom_point(data = mcycle, aes(x = times, y = accel), color = "gray") +
  geom_point(color = "red") + labs(title = "degree = 1, span = 0.12",
                                   x = "time", y = "acceleration")
best_span2 <- test_models2[[15]]$loessplot +
  geom_point(data = mcycle, aes(x = times, y = accel), color = "gray") +
```

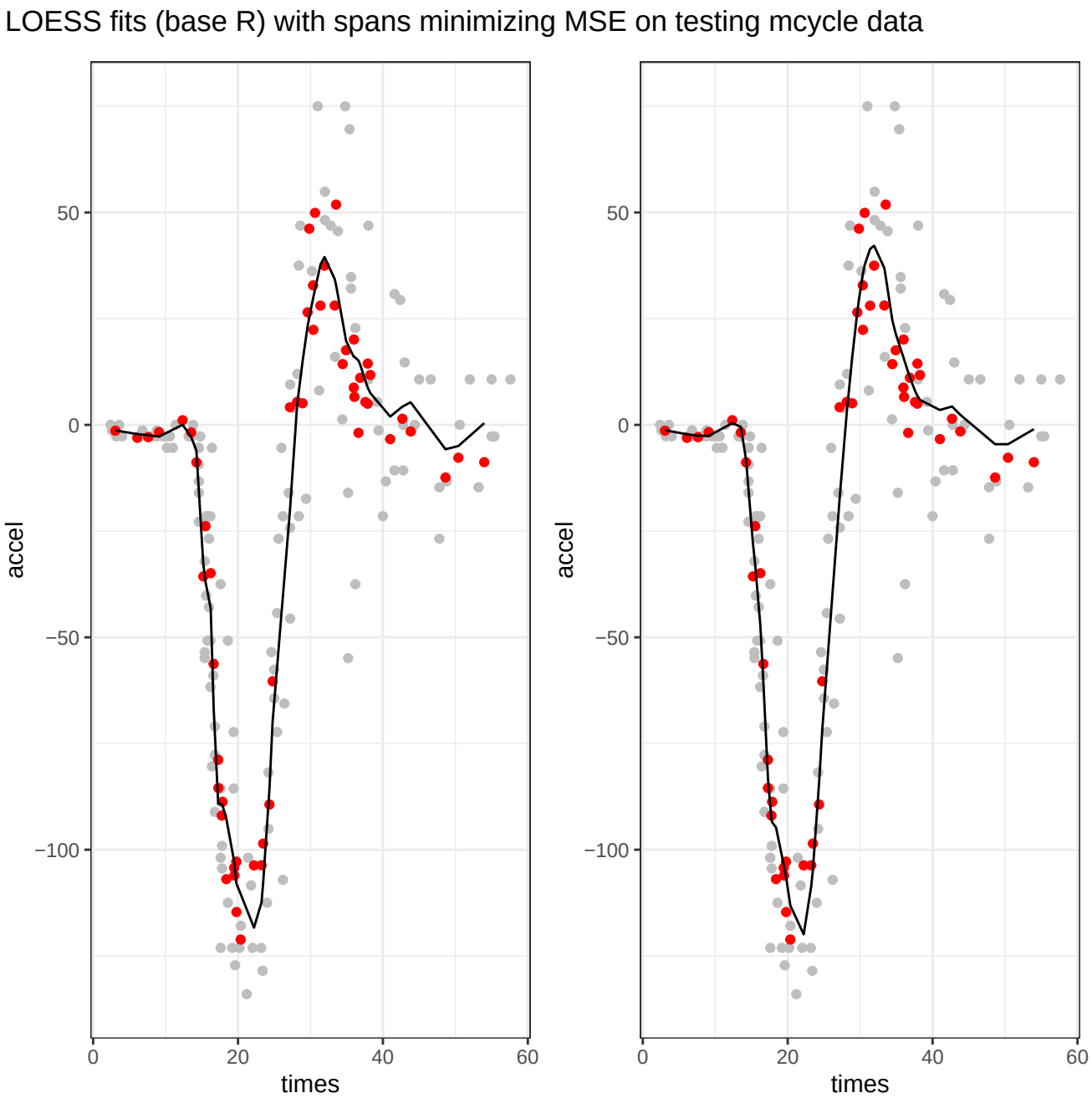
```
  geom_point(color = "red") + labs(title = "degree = 2, span = 0.21",
                                   x = "time", y = "acceleration")

(best_span1 + best_span2) + plot_annotation("LOESS fits with spans minimizing MSE on testing mcycle data")
```



We can compare our plots to the results from R's built in `loess()` function:

```
# Get fits using the base R loess function
compare_d1 <- loess(accel ~ times, data = mcycle, span = 0.12, degree = 1)
compare_d2 <- loess(accel ~ times, data = mcycle, span = 0.21)
# Predict on testing data
ypred_d1 <- predict(compare_d1, newdata = mcycle_test)
ypred_d2 <- predict(compare_d2, newdata = mcycle_test)
# Graph fits
best_span1_baser <- ggplot(data = mcycle, aes(x = times, y = accel)) + geom_point(color = "gray") +
  geom_point(data = mcycle_test, aes(x = times, y = accel), color = "red") + theme_bw() +
  geom_line(data = data.frame(mcycle_test$times, ypred_d1), aes(x = mcycle_test$times, y = ypred_d1))
best_span2_baser <- ggplot(data = mcycle, aes(x = times, y = accel)) + geom_point(color = "gray") +
  geom_point(data = mcycle_test, aes(x = times, y = accel), color = "red") + theme_bw() +
```

```
  geom_line(data = data.frame(mcycle_test$times, ypred_d2), aes(x = mcycle_test$times, y = ypred_d2))
(best_span1_baser + best_span2_baser) + plot_annotation("LOESS fits (base R) with spans minimizing MSE on test
```



Problem 3

```
# Reading in the data for this problem
data("fgl")
```

The dataset I chose is the `fgl` dataset from the `MASS` library package. This dataset contains information on the refractive index, chemical makeup, and type of 214 glass fragments collected through forensic work.

A data dictionary is provided below:

```
# Define variable types
var_types <- c("numeric", "numeric", "numeric", "numeric", "numeric", "numeric",
               "numeric", "numeric", "numeric", "Factor")
# Add a short description for each variable
var_labels <- c("Last 4 digits of refractive index",
                "Sodium (% by weight of oxide)",
```

```
                "Manganese (% by weight of oxide)",
                "Aluminum (% by weight of oxide)",
                "Silicon (% by weight of oxide)",
                "Potassium (% by weight of oxide)",
                "Calcium (% by weight of oxide)",
                "Barium (% by weight of oxide)",
                "Iron (% by weight of oxide)",
                "Glass type, classified by origin")
# Form the data dictionary as a data frame
data_dict <- data.frame(Name = names(fgl),
             Description = var_labels,
             Type = var_types,
             stringsAsFactors = FALSE)
# Nicely format data dictionary for the output
knitr::kable(
  data_dict,
  caption = "Data dictionary for fgl dataset"
)
```

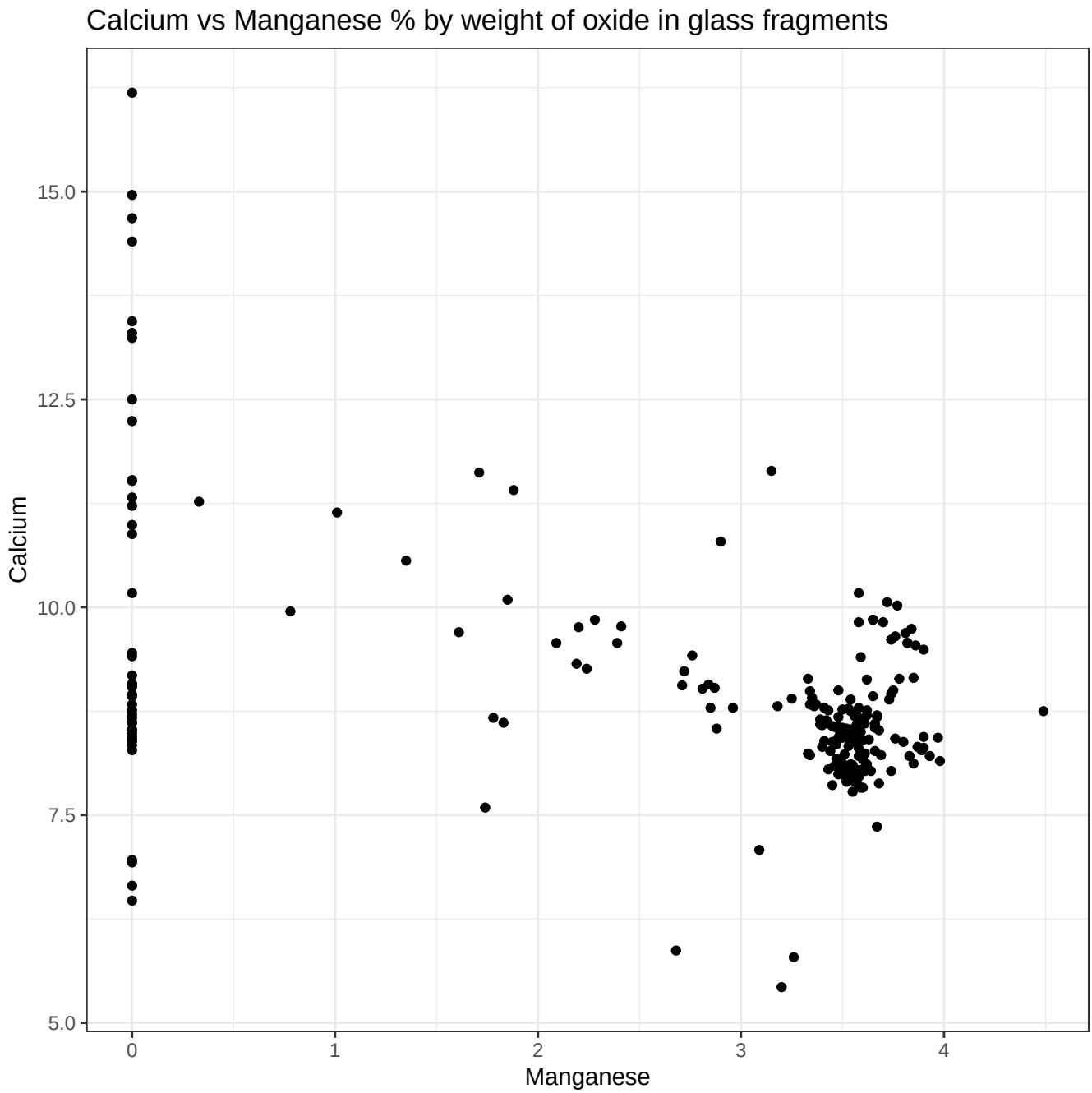
Table 6: Data dictionary for fgl dataset

Name	Description	Type
RI	Last 4 digits of refractive index	numeric
Na	Sodium (% by weight of oxide)	numeric
Mg	Manganese (% by weight of oxide)	numeric
Al	Aluminum (% by weight of oxide)	numeric
Si	Silicon (% by weight of oxide)	numeric
K	Potassium (% by weight of oxide)	numeric
Ca	Calcium (% by weight of oxide)	numeric
Ba	Barium (% by weight of oxide)	numeric
Fe	Iron (% by weight of oxide)	numeric
type	Glass type, classified by origin	Factor

More information on the dataset, including factor levels for the `type` variable, can be found in the documentation for the `MASS` library package here.

In this analysis, we will focus on predicting calcium content from manganese content. A scatterplot of calcium vs manganese content is given below:

```
# Scatterplot of Ca vs Mg
ggplot(data = fgl, aes(x = Mg, y = Ca)) + geom_point() + theme_bw() +
  labs(title = "Calcium vs Manganese % by weight of oxide in glass fragments",
       x = "Manganese", y = "Calcium")
```



For fragments that had none-zero levels of both calcium and manganese, the relationship looks to be negative and somewhat linear, perhaps with a slight curve towards the right of the plot. However, note that there is a large group observations with 0% manganese content by weight of oxide, and that these observations also tend to have high levels of calcium. Thus, it is plausible that the true relationship may be a very steep curve for manganese contents close to 0% by weight of oxide while being more linear (or at least less curved) for higher manganese contents.

We'll use K-fold cross validation with $K = 10$ to find the optimal span values for local fits with degree = 1 and for degree = 2. We'll define the span sequences differently for each degree, as degree = 2 fits will require higher span values to ensure that there are always enough unique points in the subset of the training data to fit the quadratic model; however, both span sequences will be incremented by 0.01 and cover the same amount of different possible spans. We'll then split the indicess of the data into 10 folds. Before starting the cross validation, we'll define $K \times n$ matrices to store our computed MSEs, where each row corresponds to a different fold as testing data and each column corresponds to a different possible span value.

For each fold, we'll split the larger data set into training and testing data. Then, for each possible span, we'll fit the corresponding model of the right degree, and store the MSE in the corresponding matrix. Note that because the spans we're checking at each iteration of the inner loop will differ. For example, when $j = 1$, we do LOESS for degree = 1 and span = 0.21 and for degree = 1 and span = 0.31. However, because we check the same amount of possible spans for each degree, there aren't any issues with one degree being done before the other. After the loops have concluded, we take the mean of each column in the MSE matrix to get the mean MSE for each span across all folds. `which.min()` is used to determine the

column index of the minimum average MSE, and we can index into the corresponding array of spans for that degree to find the optimal span.

```
# For reproducibility
set.seed(329)
k <- 10
n <- nrow(fgl)
# Different span values for each degree, as to avoid errors where there
# aren't more than degree number of unique x-values in a window
spans_d1 <- seq(0.21, 0.60, by = 0.01)
spans_d2 <- seq(0.31, 0.70, by = 0.01)
# Make folds
folds <- split(sample(1:n), rep(1:k, length = n))
# Make MSE matrices, where each row corresponds to a fold
# and each column corresponds to a different span
MSE_d1 <- matrix(NA, nrow = k, ncol = length(spans_d1))
MSE_d2 <- matrix(NA, nrow = k, ncol = length(spans_d2))
# For each fold, separate out training and testing data
for (i in 1:k){
  testing_indices <- folds[[i]]
  training_data <- fgl[-testing_indices, c("Mg", "Ca")]
  testing_data <- fgl[testing_indices, c("Mg", "Ca")]
  # For each span, fit the model and record MSE
  for (j in 1:length(spans_d1)) {
    fit1 <- myloess(x_train = training_data$Mg, y_train = training_data$Ca,
                    x_test = testing_data$Mg, y_test = testing_data$Ca,
                    span = spans_d1[j], degree = 1, show.plot = FALSE)
    MSE_d1[i, j] <- fit1$MSE
    fit2 <- myloess(x_train = training_data$Mg, y_train = training_data$Ca,
                    x_test = testing_data$Mg, y_test = testing_data$Ca,
                    span = spans_d2[j], degree = 2, show.plot = FALSE)
    MSE_d2[i, j] <- fit2$MSE
  }
}
# Get mean MSEs for each span
mean_MSE_d1 <- colMeans(MSE_d1)
mean_MSE_d2 <- colMeans(MSE_d2)
# Find optimal span for each degree, based on which has the lowest
# corresponding average MSE across all folds
optim_span_d1 <- spans_d1[which.min(mean_MSE_d1)]
optim_span_d2 <- spans_d2[which.min(mean_MSE_d2)]
# Print optimal spans
optim_span_d1
```

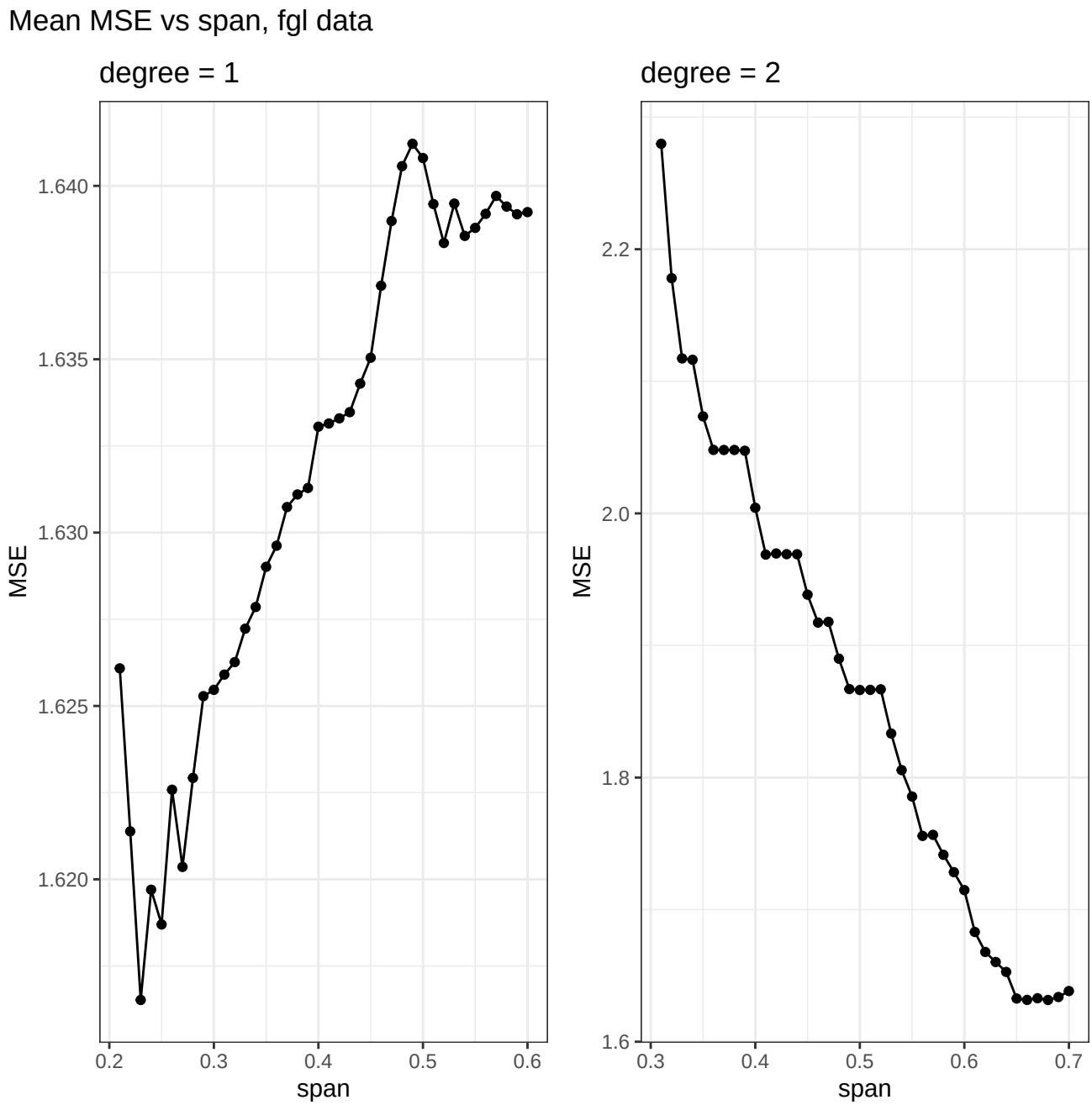
```
[1] 0.23
```

```
optim_span_d2
```

```
[1] 0.68
```

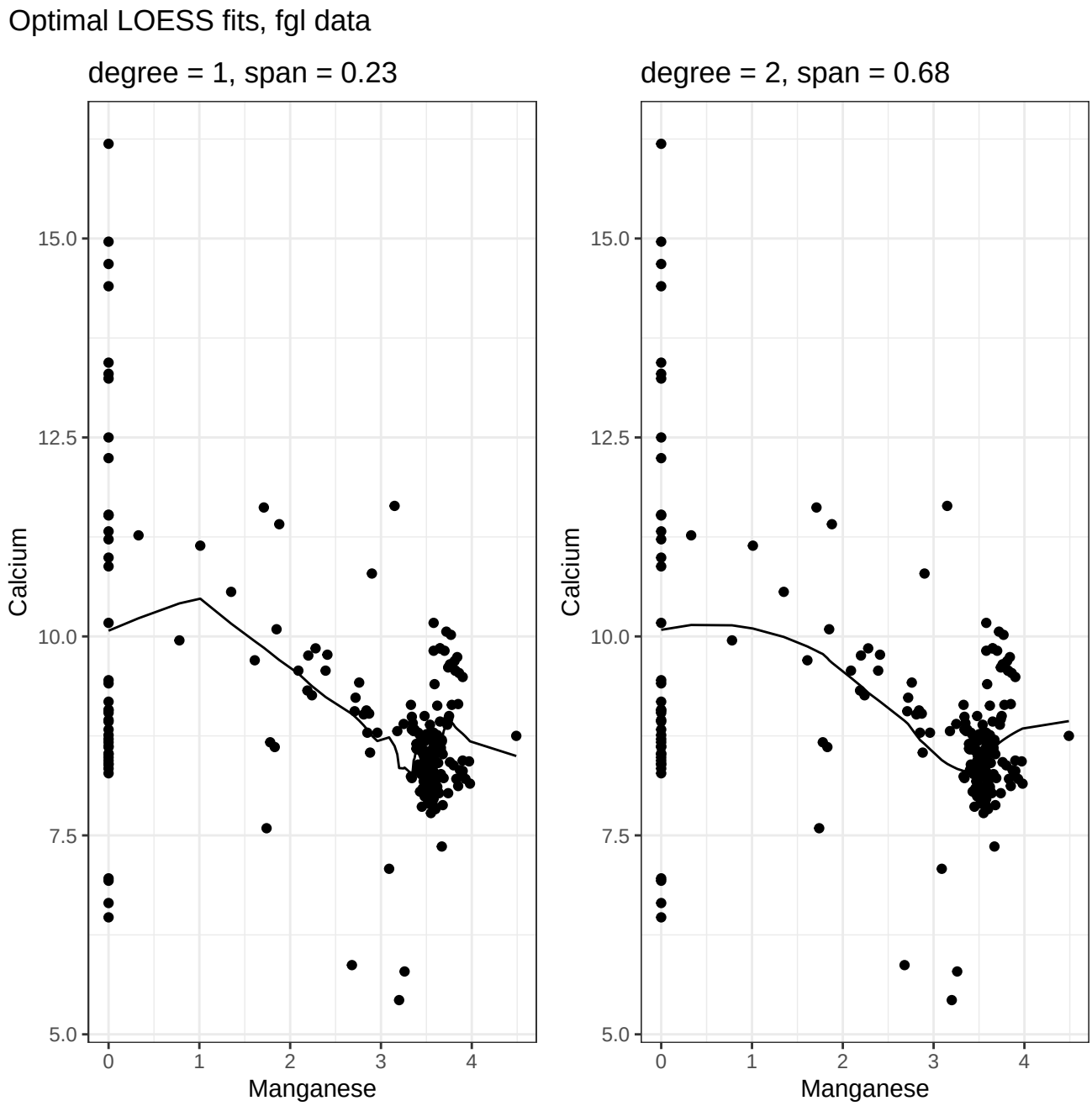
For degree = 1, the optimal span is 0.23. For degree = 2, the optimal span is 0.68. We can graph the relationship between span and average MSE for each degree:

```
# Bundle span and mean MSEs for each degree
spans_df1 <- data.frame(span = spans_d1, MSE = mean_MSE_d1)
spans_df2 <- data.frame(span = spans_d2, MSE = mean_MSE_d2)
# Graph mean MSE vs span for each degree
span_curve_d1 <- ggplot(data = spans_df1, aes(x = span, y = MSE)) + geom_point() +
  geom_line(aes(group = 1)) + theme_bw() + labs(title = "degree = 1")
span_curve_d2 <- ggplot(data = spans_df2, aes(x = span, y = MSE)) + geom_point() +
  geom_line(aes(group = 1)) + theme_bw() + labs(title = "degree = 2")
span_curve_d1 + span_curve_d2 + plot_annotation(title = "Mean MSE vs span, fgl data")
```



Interestingly, for both degree = 1 and degree = 2, the minimum MSE is around 1.6. We can also plot the LOESS fits with the optimal span for each degree:

```
optim_d1 <- myloess(x_train = fgl$Mg, y_train = fgl$Ca, span = 0.23, degree = 1,
                    show.plot = FALSE)
optim_d2 <- myloess(x_train = fgl$Mg, y_train = fgl$Ca, span = 0.68, degree = 2,
                    show.plot = FALSE)
optim_d1_plot <- optim_d1$loessplot + labs(title = "degree = 1, span = 0.23",
                                           x = "Manganese", y = "Calcium")
optim_d2_plot <- optim_d2$loessplot + labs(title = "degree = 2, span = 0.68",
                                           x = "Manganese", y = "Calcium")
optim_d1_plot + optim_d2_plot + plot_annotation(title = "Optimal LOESS fits, fgl data")
```



Interestingly, the optimal fits look a lot different than I though they would be. Much of the curvature is actually towards the right of the plot for relatively higher concentrations of manganese; there's a slight decrease in the predicted calcium content for glass fragments with that are around 3.5% manganese by weight of oxide. By contrast, when the manganese content is near 0, the curve is relatively flat. I would conjecture that if the calcium concentrations for glass fragments that are 0% manganese weren't so spread out (i.e., if they were more uniformly lower or higher compared to glass fragments that did contain some manganese), there would be more curvature in that area like I was expecting. As it is, I do believe that the LOESS fits are better than those of a standard linear or polynomial model, as the relationship does have some slight curvature that makes it not linear, but the curvature isn't strong enough for a polynomial model to completely capture.

Animated LOESS

Part 1

Using the 'gganimate' library package, we wrote an animated version of the LOESS function. Calling the function we made that animates the function- 'animated_loess', we use the function upon the 'mycycle' dataset.

```
# Load the motorcycle crash dataset
data(mcycle, package = "MASS")
```

```r
# Generate animations for different span values (0.2, .33, .5, and .66)
#animated_loess(mcycle$times, mcycle$accel, span = 0.2,  filename = "mcycle_span_0.2.gif")
#animated_loess(mcycle$times, mcycle$accel, span = 0.33, filename = "mcycle_span_0.33.gif")
#animated_loess(mcycle$times, mcycle$accel, span = 0.5,  filename = "mcycle_span_0.5.gif")
#animated_loess(mcycle$times, mcycle$accel, span = 0.66, filename = "mcycle_span_0.66.gif")
```

After generating the gif files, each gif file shows us an animation and the results differ from each gif depending on the assigned weight given. We generated 4 animations based on different span values- 0.2, 0.33, 0.5, and 0.66. The best span value where the curvature of line of the function best follows closest to the data points positioning is span = 0.2. What we notice is the the greater in value the span is- the more offset and further the curvature of the line function is from the data points, thus the more unprecise the curves are.

Problem 4

```r
Auto_new <- Auto[, -9]
newOrigin <- c("USA", "European", "Japanese")
Auto_new$origin <- factor(newOrigin[Auto_new$origin], newOrigin)

# preprocessing

set.seed(67)

# Separate X and Y
X <- Auto_new[, setdiff(names(Auto_new), "origin")]
y <- Auto_new$origin

# standardize to normal scale since knn is distance based
X_scaled <- scale(X)

# 70/30 traning testing data split
n <- nrow(X_scaled)
train_split <- sample(1:n, size = 0.7 * n)
X_train <- X_scaled[train_split, ]
X_test  <- X_scaled[-train_split, ]
y_train <- y[train_split]
y_test  <- y[-train_split]
```

kNN

Part 2

```r
# try various k values
k_values <- seq(1, 10, by = 2)

# storage for accuracy
acc_knn <- numeric(length(k_values))

# apply for each k value and record accuracy
for(i in seq_along(k_values)){

  k <- k_values[i]

  res_knn <- mykNN(X_train, X_test, y_train, y_test,
                   k = k, weighted = FALSE)

  acc_knn[i] <- res_knn$accuracy
}
```

Part 3

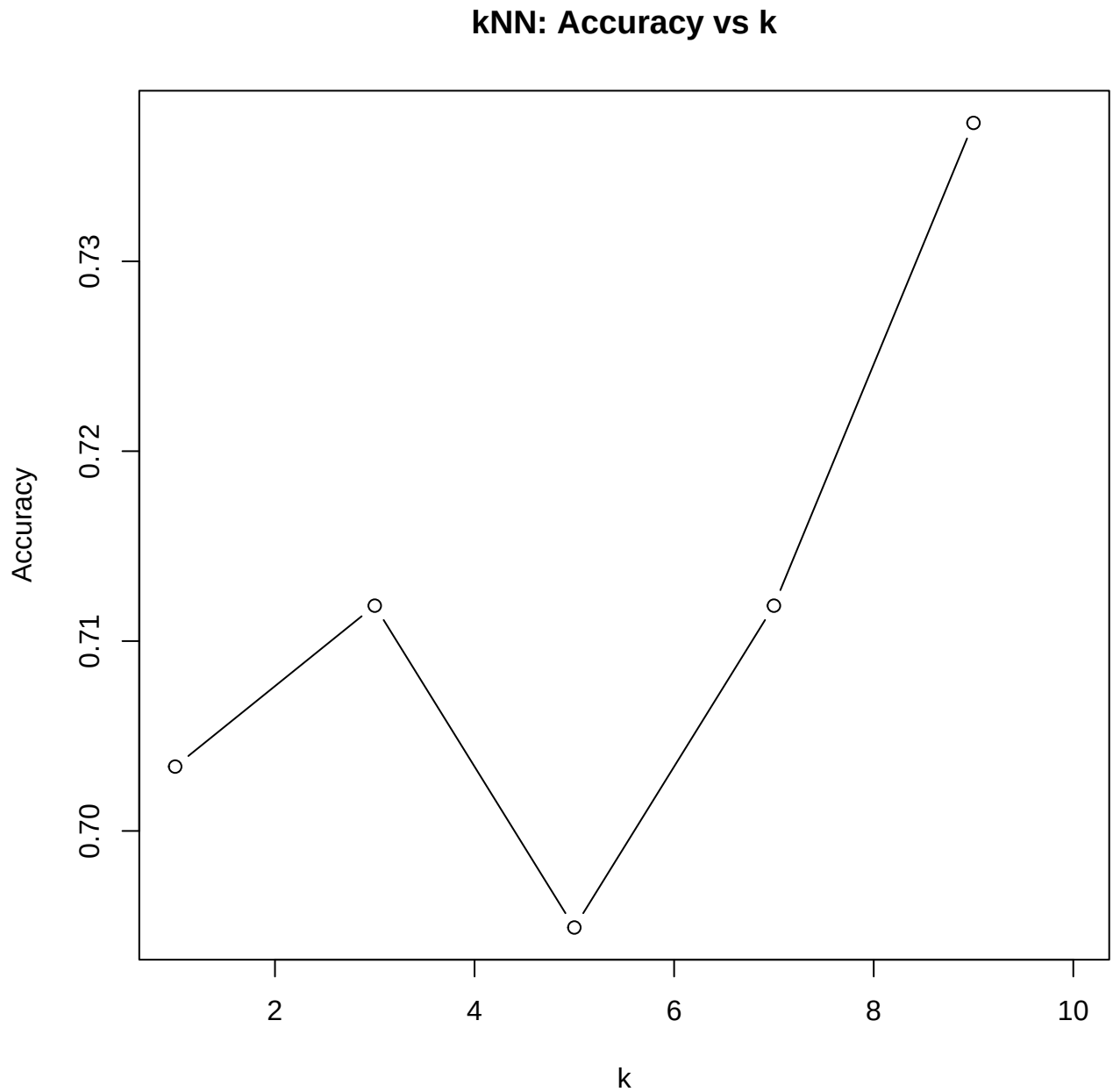
```
knn_table <- data.frame(
  k = k_values,
  Accuracy = acc_knn
)

kable(knn_table, digits = 3, caption = "kNN Accuracy for Different k")
```

Table 7: kNN Accuracy for Different k

k	Accuracy
1	0.703
3	0.712
5	0.695
7	0.712
9	0.737

Part 4

```
plot(k_values, acc_knn, xlab = "k", xlim=c(1,10), type='b', ylab = "Accuracy",
     main = "kNN: Accuracy vs k")
```



From the plot we can see that the best number of neighbors to use for kNN is 9.

Part 5

```
best_k_knn <- k_values[which.max(acc_knn)]

final_knn <- mykNN(X_train, X_test, y_train, y_test,
                   k = best_k_knn, weighted = FALSE)

best_k_knn
```

```
[1] 9
```

```
final_knn$accuracy
```

```
[1] 0.7372881
```

```
final_knn$confusion_matrix
```

```
          Actual
Predicted  USA European Japanese
  USA       64       10        5
  European   0        6        4
```

```
  Japanese   6        6       17
```

dnkNN

Part 2

```r
# same process as kNN chunk
acc_dwkNN <- numeric(length(k_values))

for(i in seq_along(k_values)){

  k <- k_values[i]

  res_dwkNN <- mykNN(X_train, X_test, y_train, y_test,
                     k = k, weighted = TRUE)

  acc_dwkNN[i] <- res_dwkNN$accuracy
}
```

Part 3

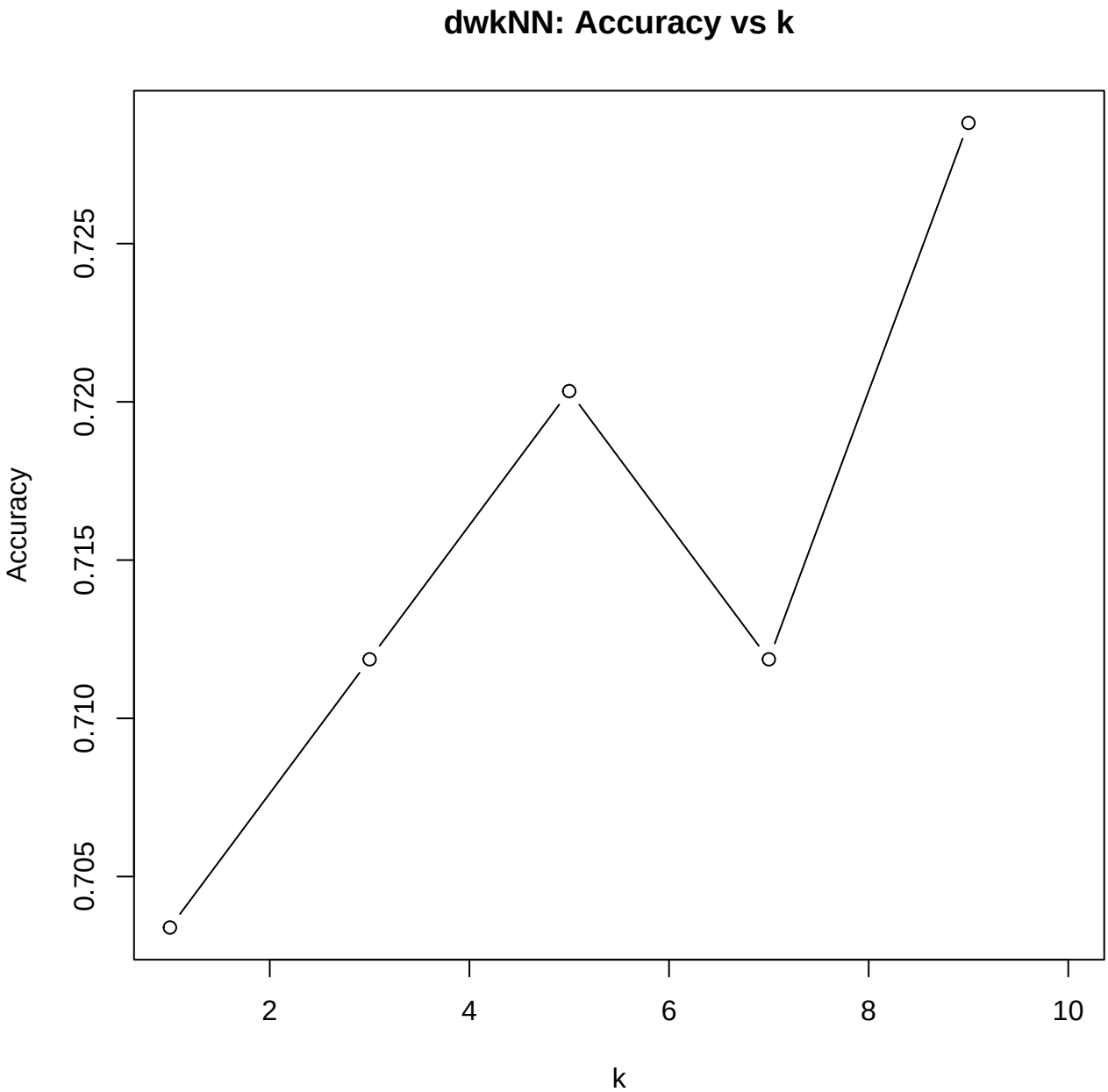
```r
dwknn_table <- data.frame(
  k = k_values,
  Accuracy = acc_dwkNN
)

kable(dwknn_table, digits = 3, caption = "dwkNN Accuracy for Different k")
```

Table 8: dwkNN Accuracy for Different k

k	Accuracy
1	0.703
3	0.712
5	0.720
7	0.712
9	0.729

Part 4

```r
plot(k_values, acc_dwkNN, xlab = "k", xlim=c(1,10), type='b', ylab = "Accuracy",
     main = "dwkNN: Accuracy vs k")
```



From the plot we can see that the best number of neighbors to use for dwkNN is 9.

Part 5

```
best_k_dwkNN <- k_values[which.max(acc_dwkNN)]

final_dwkNN <- mykNN(X_train, X_test, y_train, y_test,
                     k = best_k_dwkNN, weighted = TRUE)

best_k_dwkNN
```

```
[1] 9
```

```
final_dwkNN$accuracy
```

```
[1] 0.7288136
```

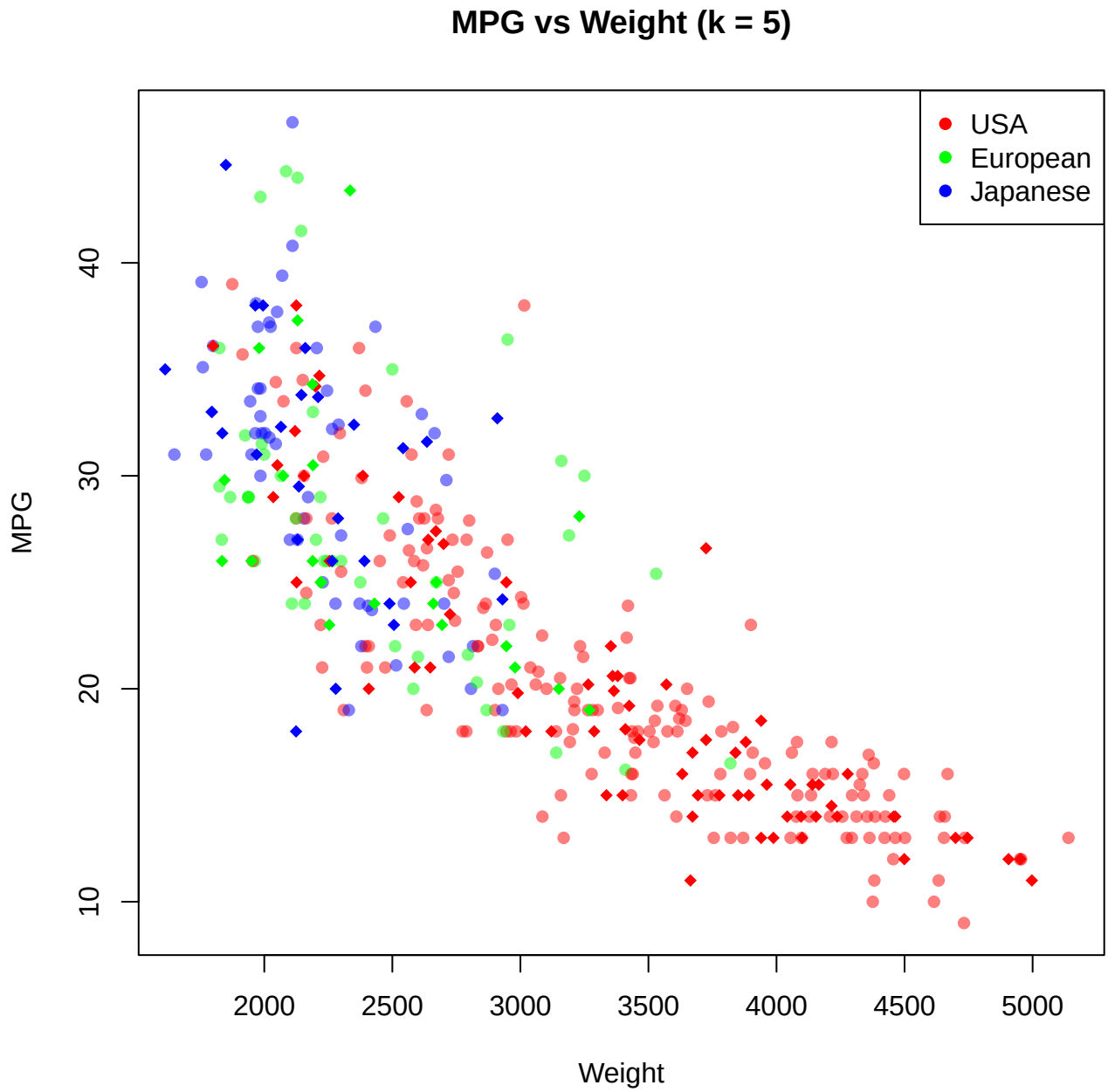
```
final_dwkNN$confusion_matrix
```

```
          Actual
Predicted  USA European Japanese
  USA       62        7        5
  European   0        7        4
```

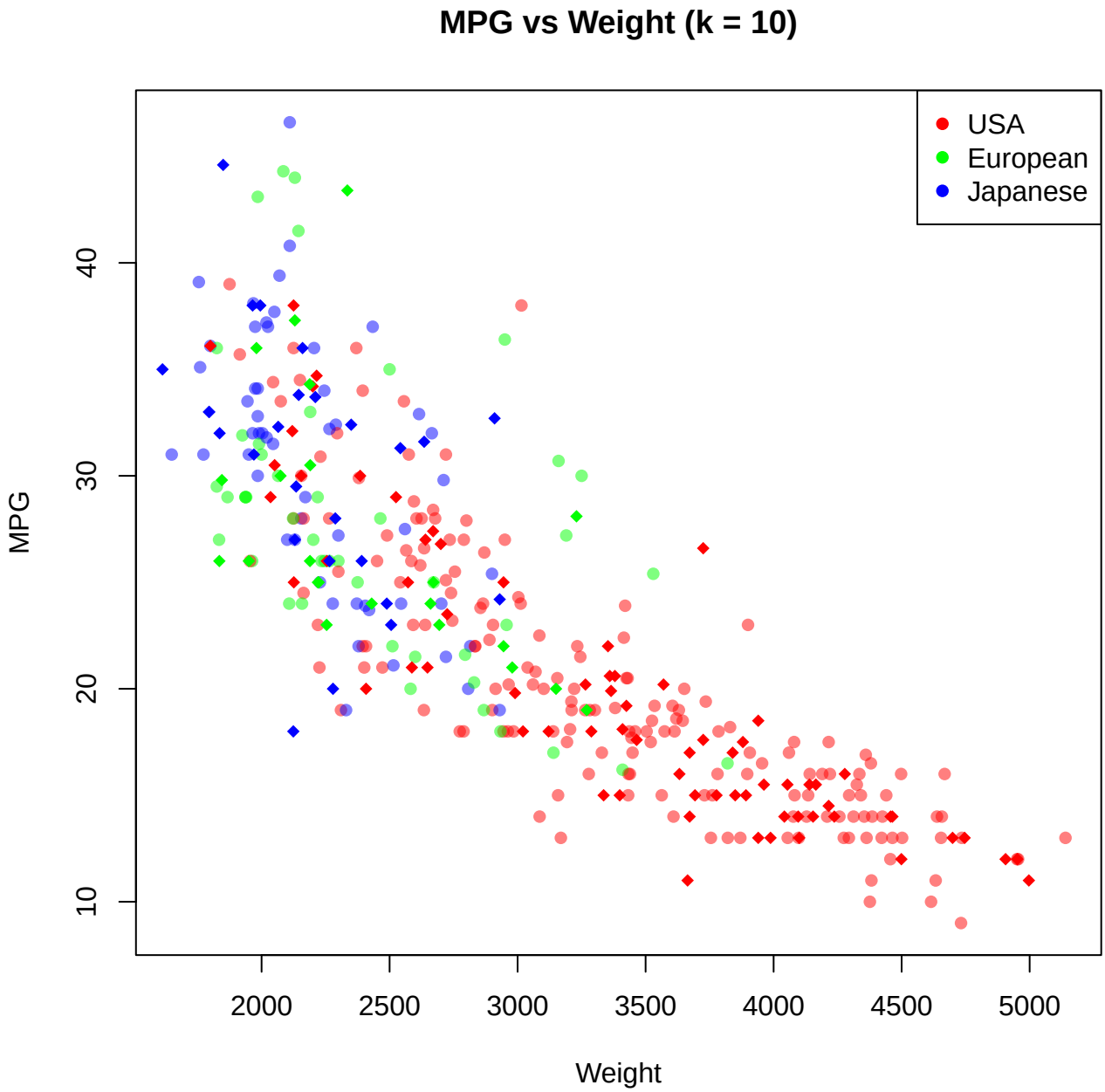
```
  Japanese   8        8       17
```

Overall, results are similar between kNN and dwkNN. In particular they both have highest accuracies at k=9. One thing to note though is that for k=5 accuracy is significantly higher for dwkNN due to it assigning more weight to closer neighbors.

```r
cols <- c("red", "green", "blue")
train_colors <- cols[as.numeric(y_train)]
test_colors  <- cols[as.numeric(y_test)]

# raw unscaled data
X_train_raw <- Auto_new[train_split, ]
X_test_raw  <- Auto_new[-train_split, ]

plot(X_train_raw$weight, X_train_raw$mpg,
     col = adjustcolor(train_colors, alpha.f = 0.5),
     pch = 16,
     xlab = "Weight",
     ylab = "MPG",
     main = "MPG vs Weight (k = 5)")

# testing data points
points(X_test_raw$weight, X_test_raw$mpg,
       col = test_colors,
       pch = 18)  # solid diamonds

# legend
legend("topright",
       legend = levels(y_train),
       col = cols, pch=16)
```




```
plot(X_train_raw$weight, X_train_raw$mpg,
     col = adjustcolor(train_colors, alpha.f = 0.5),
     pch = 16,
     xlab = "Weight",
     ylab = "MPG",
     main = "MPG vs Weight (k = 10)")

# testing data points
points(X_test_raw$weight, X_test_raw$mpg,
       col = test_colors,
       pch = 18)

# legend
legend("topright",
       legend = levels(y_train),
       col = cols,
       pch = 16)
```



Problem 5

Part a

Part b

Problem 6